



Milestone #2

mini Database Management System Development

Reported By:

GROUP MahSQL.5

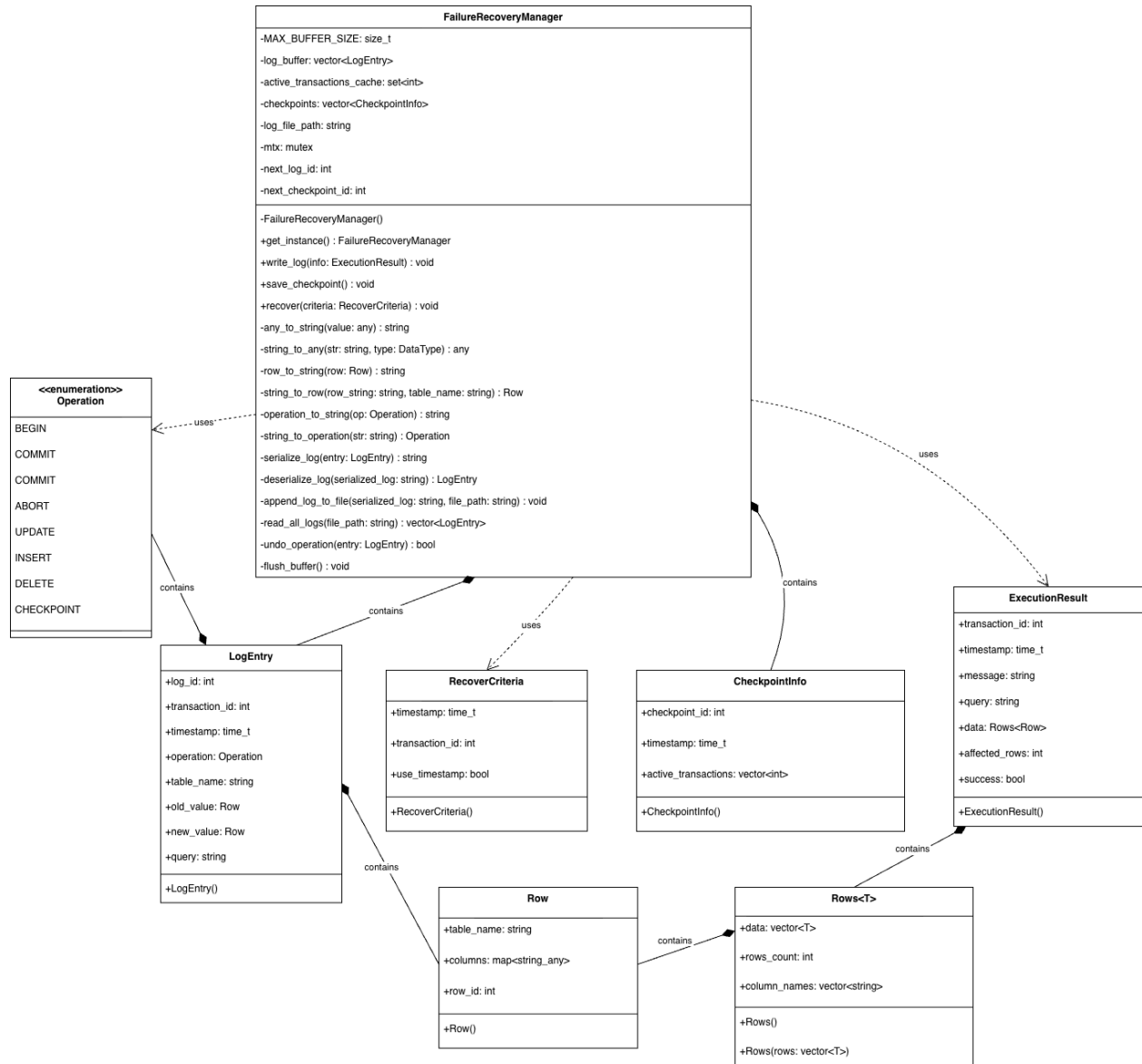
assembly

Anggota:

1. Alvin Christopher Santausa - 13523033
2. Muhammad Iqbal Haidar - 13523111
3. Ferdin Arsenarendra Purtadi - 13523117
4. Reza Ahmad Syarif - 13523119

Part I. Gambaran Umum Komponen

Berikut *class diagram* Failure Recovery Manager



Gambar *Class Diagram* Failure Recovery Manager

Kelas	Kode	Fungsi
FailureRecover yManager	<pre> class FailureRecoveryManager { public: static FailureRecoveryManager& </pre>	Kelas ini berfungsi sebagai pusat kendali proses pencatatan log, pembuatan checkpoint, dan

	<pre> get_instance(); FailureRecoveryManager(const FailureRecoveryManager&) = delete; FailureRecoveryManager& operator=(const FailureRecoveryManager&) = delete; ~FailureRecoveryManager(); void write_log(const ExecutionResult& info); void save_checkpoint(); void recover(const RecoverCriteria& criteria); private: FailureRecoveryManager(); const size_t MAX_BUFFER_SIZE = 50; std::vector<LogEntry> log_buffer; std::set<int> active_transactions_cache; std::vector<CheckpointInfo> checkpoints; std::string log_file_path; std::mutex mtx; int next_log_id; int next_checkpoint_id; std::string any_to_string(const std::any& value); std::any string_to_any(const std::string& str, DataType type); std::string row_to_string(const Row& row); Row string_to_row(const std::string& row_string, const std::string& table_name); std::string operation_to_string(Operation op); Operation string_to_operation(const std::string& str); </pre>	pemulihan. Kelas FailureRecoveryManager memiliki beberapa method yaitu: 1. get_instance() : Mengembalikan single instance dari FailureRecoveryManager (Singleton pattern). 2. write_log(info: ExecutionResult) : Mencatat operasi database ke log buffer. Menentukan tipe operasi (INSERT/UPDATE/DELETE/BEGIN/COMMIT/ABORT), menyimpan old_value dan new_value, lalu menambahkan ke buffer. Memanggil flush jika buffer mencapai kapasitas maksimum. 3. save_checkpoint() : Membuat checkpoint dengan melakukan flush semua buffer ke disk, mencatat transaksi aktif, dan menyimpan checkpoint info untuk recovery point. 4. recover(criteria: RecoverCriteria) : Melakukan recovery dengan UNDO operasi secara backward berdasarkan transaction_id. Membaca semua log dari disk, filter berdasarkan kriteria, lalu rollback operasi dari yang terbaru ke awal. 5. any_to_string(value: any) : Mengkonversi `std::any` menjadi string untuk serialisasi (mendukung tipe int, float, dan string). 6. string_to_any(str: string, type: DataType) : Mengkonversi string kembali menjadi `std::any` berdasarkan tipe data yang ditentukan. 7. row_to_string(row: Row) : Melakukan serialisasi Row menjadi
--	--	--

	<pre> std::string serialize_log(const LogEntry& entry); LogEntry deserialize_log(const std::string& serialized_log); void append_log_to_file(const std::string& serialized_log, const std::string& file_path); std::vector<LogEntry> read_all_logs(const std::string& file_path); bool undo_operation(const LogEntry& entry); void flush_buffer(); }; </pre>	format string: `ROW_ID#{attr1:val1,attr2:val2,...}` 8. string_to_row(row_string: string, table_name: string) : Melakukan deserialisasi string kembali menjadi Row object dengan table_name yang sesuai. 9. operation_to_string(op: Operation) : Mengkonversi enum Operation menjadi string (BEGIN/COMMIT/ABORT/INSERT/UPDATE/DELETE/CHECKPOINT). 10. string_to_operation(str: string) : Mengkonversi string kembali menjadi enum Operation. 11. serialize_log(entry: LogEntry) : Memformat LogEntry menjadi string dengan delimiter ` ` dalam format: LOG_ID TX_ID TIMESTAMP OPERATION TABLE OLD_VALUE NEW_VALUE QUERY 12. deserialize_log(serialized_log: string) : Melakukan parsing string log kembali menjadi LogEntry object dengan validasi error handling. 13. append_log_to_file(serialized_log : string, file_path: string) : Menulis satu baris log ke file dengan mode append untuk persistence. 14. read_all_logs(file_path: string) : Membaca semua log entries dari file dan mengembalikan vector berisi LogEntry dengan error handling untuk file tidak ditemukan atau corrupt. 15. flush_buffer() : Menulis semua log entries di buffer ke disk file, lalu mengosongkan buffer untuk menghemat memory.
--	---	---

		16. undo_operation(entry: LogEntry) : Melakukan UNDO operasi berdasarkan tipe: - INSERT → DELETE row baru (new_value) - DELETE → INSERT kembali row lama (old_value) - UPDATE → Restore ke nilai lama (old_value)
LogEntry	<pre> struct LogEntry { int log_id; int transaction_id; std::time_t timestamp; Operation operation; std::string table_name; Row old_value; Row new_value; std::string query; LogEntry() : log_id(-1), transaction_id(-1), timestamp(std::time(nullptr)) {} }; </pre>	Kelas ini merepresentasikan satu entri log yang disimpan saat suatu transaksi melakukan operasi.
CheckpointInfo	<pre> struct CheckpointInfo { int checkpoint_id; std::time_t timestamp; std::vector<int> active_transactions; CheckpointInfo() : checkpoint_id(-1), timestamp(std::time(nullptr)) {} }; </pre>	Merepresentasikan checkpoint yang disimpan untuk mempercepat proses recovery.
RecoveryCriteria	<pre> struct RecoverCriteria { std::time_t timestamp; int transaction_id; bool use_timestamp; RecoverCriteria() : timestamp(0), transaction_id(-1), use_timestamp(false) {} }; </pre>	Kelas ini digunakan untuk menentukan bagaimana proses recovery dilakukan.

Operation	<pre>enum class Operation { BEGIN, COMMIT, ABORT, UPDATE, INSERT, DELETE, CHECKPOINT };</pre>	Enumerasi untuk mendefinisikan jenis operasi yang dicatat dalam log
Row	<pre>struct Row { std::string table_name; std::map<std::string, std::any> columns; int row_id; Row() : row_id(-1) {} };</pre>	Untuk merepresentasikan satu baris data dalam tabel

Part II. Bagian yang Telah Selesai

Berikut adalah bagian yang berhasil dikembangkan

- Inisialisasi tipe dan struktur data yang diperlukan untuk Failure Recovery
- Pembuatan Class Diagram
- Konstruksi method write_log
- Konstruksi method save_checkpoint
- Konstruksi method recover
- Konstruksi fitur Penyimpanan Log
- Konstruksi fitur Recovery

Part III. Bagian yang Belum Selesai

Berikut tugas-tugas yang masih perlu diselesaikan :

- Integrasi dengan Storage Manager untuk undo transaksi
- Implementasi System Failure Recovery
- Implementasi write log untuk bagian update dan delete (masih menggunakan placeholder)

Part IV. Distribusi Workload

NIM	Nama	Workload
13523033	Alvin Christopher Santausa	Pembuatan laporan, implementasi <code>recover(RecoverCriteria)</code> , logika Rollback, dan mekanisme pembacaan log secara backward.
13523111	Muhammad Iqbal Haidar	Pembuatan laporan, membuat fungsi konversi <code>row_to_string</code> dan <code>string_to_row</code> , implementasi fungsi <code>serialize_log</code> , implementasi fungsi <code>deserialize_log</code> , menangani operasi File Stream ke <code>log_file_path</code> .
13523117	Ferdin Arsenarendra Purtadi	Pembuatan laporan, implementasi fungsi <code>write_log</code> , menerjemahkan <code>ExecutionResult</code> dari Query Processor menjadi <code>logEntry</code> .
13523119	Reza Ahmad Syarif	Pembuatan laporan, pembuatan kerangka Singleton untuk <code>FailureRecoveryManager</code> , mengelola <code>log_buffer</code> , implementasi logika <code>save_checkpoint</code> , dan thread safety.

Part V. Lampiran

Sertakan lampiran yang diperlukan. Lampiran wajib adalah **tautan *GitHub Release*** untuk setiap *checkpoint*.

Github Release: [BP04/IF3140_DBMS](#)

Find or start a conversation

Friends

Message Requests

Nitro

Shop

Quests

Direct Messages

Arsenaferdin, Iqbal Haidar, Rejaah

Bad Cards

Arsenaferdin

Rejaah

Iqbal Haidar

leminerale, osaker

denial

AvenueX

osaker

D2V6

Pancake

MEE6

Skimp Jr

Bloxlink

Arsenaferdin, Iqbal Haidar, Rejaah

region Automatic

Voice Connected

Arsenaferdin, Iqbal Haidar, Rejaah

Alvin

invisible

Message Arsenaferdin, Iqbal Haidar, Rejaah

Members—4

Rejaah

Alvin

Arsenaferdin

Iqbal Haidar

Direct Messages

Arsenaferdin, Iqbal Haidar, Rejaah

region Automatic

Voice Connected

Arsenaferdin, Iqbal Haidar, Rejaah

Alvin

invisible

Message Arsenaferdin, Iqbal Haidar, Rejaah

Members—4

Rejaah

Alvin

Arsenaferdin

Iqbal Haidar

https://markdownlivepreview.com/

Markdown Live Preview

This is the online markdown editor with live preview.

Members—4

Rejaah

Alvin

Arsenaferdin

Iqbal Haidar